МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод бисекции.

Студент гр. 4342 Закаблук А.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 5. Функция $f(x) = arctg(1/x) - x^2$.

Изучить метод бисекции для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Теоретический минимум.

Если найден отрезок $[a, b]$, такой, что $f(a) * f(b) < 0$, существует точка cc, в которой значение функции равно нулю, т.е. $f(c) = 0, c(a, b)$. Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс построения последовательности отрезков позволяет найти нуль функции $f(x)$ (корень уравнения $f(x) = 0$ с любой заданной точностью).

Рассмотрим один шаг итерационного процесса. Пусть на $(n - 1)$ шаге найден отрезок $[a_{n-1}, b_{n-1}] \subset [a, b]$, такой, что $f\ (a_{n-1}) * f\ (b_{n-1}) < 0$. Разделим его пополам точкой $\xi = (a_{n-1} + b_{n-1})/2$ и вычислим $f(\xi)$. Если $f(\xi) = 0$, то $\xi = (a_{n-1} + b_{n-1})/2$ корень уравнения. Если $f(\xi) \neq 0$, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

$a_n = a_{n-1}, b_n = \xi$, если $f(\xi) * f(a_{n-1}) < 0$;

$a_n = \xi, b_n = b_{n-1}$, если $f(\xi) * f(a_{n-1}) > 0$.

Если требуется найти корень с точностью ε, то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше 2ε. Тогда координата середины отрезка есть значение корня с требуемой точностью.

Метод бисекции является простым и надежным методом поиска простого корня уравнения $f(x) = 0$ (простым называется корень $x = c$ дифференцируемой функции $f(x)$, если $f(c)$ и $f'(c)$). Этот метод сходится для любых непрерывных функций $f(x)$, в том числе недифференцируемых.

Скорость его сходимости невысока. Для достижения точности необходимо совершить $N \approx log_2(b-a)/\varepsilon$ итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

Выполнение работы.

1) Графический способ. С помощью графического веб-калькулятора desmos.com построим график (Рисунок 1) для определения отрезка $[a, b]$ близкого к $f(x) = 0$ на котором выполняется условие $f(a) * f(b) < 0$.

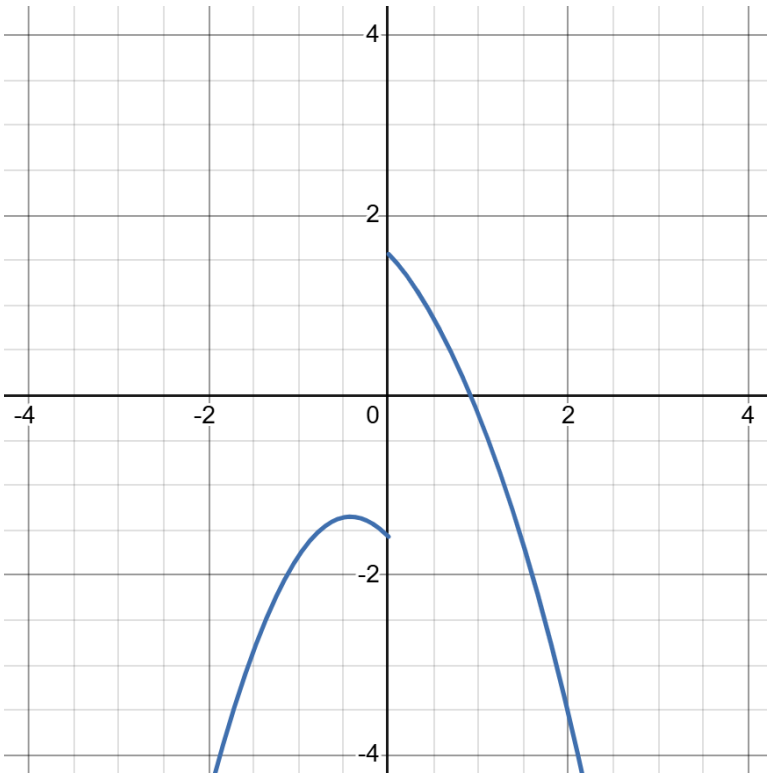

Рисунок 1 - график функции $f(x) = arctg(1/x) - x^2$

На графике видно, что корень находится около значения $x \approx 0.915$. Можно взять отрезок, где a∈(0.1,0.91), а b∈(0.92,2.0). Для простоты и надежности возьмем отрезок [0.5,1.5]. Проверка: f(0.5)≈0.857>0, f(1.5)≈−1.662<0. Условие выполняется.

Аналитический способ (аналитическое отделение корня):

Для доказательства наличия и единственности корня на выбранном отрезке исследуем функцию аналитически с помощью производной. Найдем первую производную функции: f `(x) = -1/(x^2+1) – 2x.

Рассмотрим поведение производной на интервале (0,+∞). Очевидно, что при любых x>0 выражения $(-1/(x^2+1))$ и $(-2x)$ строго меньше нуля. Следовательно, $f'(x) < 0$ при $x \in (0,+\infty)$. Так как производная знакопостоянна и отрицательна, функция f(x) монотонно и строго убывает на всем интервале. Это гарантирует, что функция может пересечь ось Ox не более одного раза (корень единственный).

Проверим выполнение условий теоремы Коши (Больцано-Коши) на границах выбранного отрезка [a,b]=[0.5,1.5]:

- $f(0.5)=\mathrm{arctg}(1/0.5)-0.5^2=\mathrm{arctg}(2)-0.25\approx 1.107-0.25=0.857$
- $f(1.5)=\mathrm{arctg}(1/1.5)-1.5^2=\mathrm{arctg}(2/3)-2.25\approx 0.588-2.25=-1.662$

Поскольку функция f(x) непрерывна на отрезке [0.5,1.5] и принимает на его концах значения разных знаков (f(0.5)>0, а f(1.5)<0, следовательно f(a)·f(b)<0), то на этом отрезке существует единственный корень. Аналитическое отделение корня успешно выполнено.

2) Функция `extern double F(double x)` теперь возвращает значение `std::atan(1.0, x) - (x * x)` в соответствии с вариантом $f(x) = arctg(1/x) - x^2$. Исходный код в Приложении А.
3) Создана функция, находящаяся в файле main.cpp в Приложении А.
4) Проведены вычисления, построен график (Рисунок 2).

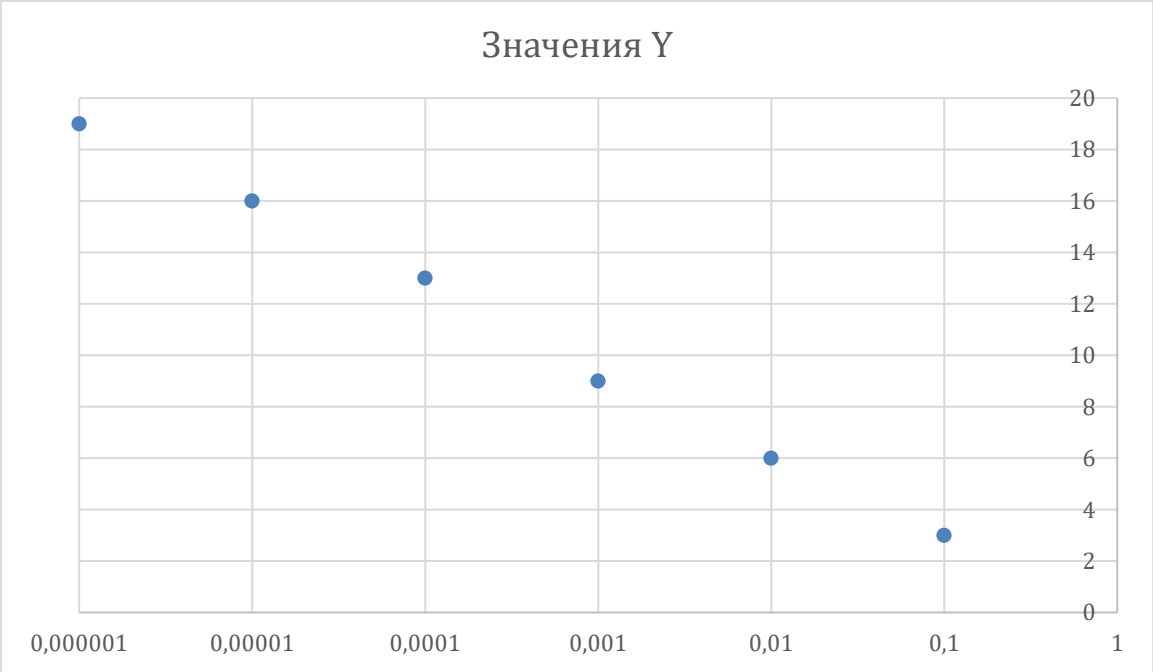

Рисунок 2 - “График” зависимости количества итераций от точности вычислений

Можно заметить, что на Рисунке 2 с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

5) Для исследования чувствительности добавлены флаги компиляции ROUND и DELTA. При ROUND=1 DELTA задается автоматически значением 0.001, если нужно можно задать отдельно. Ниже в Таблице 1 приведены значения при разных дельта.

Таблица 1 - исследование чувствительности метода к ошибкам в исходных данных

Значение DELTA	ε	Значение возвращаемое функцией BISECT	Количество итераций необходимых для заданной ε.
10^{-1}	10^{-1}	0.875	3
	10^{-2}	0.929687	6
	10^{-3}	0.915039	9
	10^{-4}	0.915039	9
	10^{-5}	0.915039	9
	10^{-6}	0.915039	9

Продолжение Таблицы 1

Значение DELTA	ε	Значение возвращаемое функцией BISECT	Количество итераций необходимых для заданной ε.
10^{-3}	10^{-1}	0.875	3
	10^{-2}	0.914062	6
	10^{-3}	0.915039	9

	10^{-4}	0.915222	13
	10^{-5}	0.915222	13
	10^{-6}	0.915222	13
10^{-5}	10^{-1}	0.875	3
	10^{-2}	0.914062	6
	10^{-3}	0.915039	9
	10^{-4}	0.915222	13
	10^{-5}	0.915215	16
	10^{-6}	0.915215	16
10^{-7}	10^{-1}	0.875	3
	10^{-2}	0.914062	6
	10^{-3}	0.915039	9
	10^{-4}	0.915222	13
	10^{-5}	0.915215	16
	10^{-6}	0.915215	19

Выводы.

В ходе работы был изучен метод половинного деления (бисекции). Для уравнения $f(x) = arctg(1/x) - x^2$ корень был успешно найден на отрезке [0.5,1.5] с точностью от 0.1 до 0.000001. Построенный график показал логарифмическую зависимость числа итераций от точности: увеличение точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод бисекции является устойчивым к малым ошибкам округления значений функции, однако при грубых возмущениях (большая Delta) процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <cmath>
#include "methods.hpp"

#ifndef LEFT
    #define LEFT 0.5 // левая граница отрезка
#endif
#ifndef RIGHT
    #define RIGHT 1.5 // правая граница отрезка
#endif

int main() {
    int iterationsCount;
    std::cout << "Eps\t\t | Root\t\t\t | Iterations\n";
    std::cout << "--------------------------------------------------\n";
    for (double epsilon = 0.1; epsilon >= 0.000001; epsilon /= 10) {
        std::cout << epsilon << "\t\t | "
                  << BISECT(LEFT, RIGHT, epsilon, iterationsCount) << "\t\t | "
                  << iterationsCount << '\n';
    }
    return 0;
}
```

Файл methods.cpp

```
#include "methods.hpp"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#ifndef ROUND_FLAG
    #define ROUND_FLAG 0
#endif
#ifndef DELTA
    #define DELTA 0.001
#endif

#if !ROUND_FLAG
extern double F(double x) {
    // Вариант 5: f(x) = arctg(1/x) - x^2
    return atan(1.0 / x) - (x * x);
}
#else
extern double F(double x) {
    return Round(atan(1.0 / x) - (x * x), DELTA);
}
#endif

double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;

    if (FLeft * FRight > 0.0) {
        puts("Неверное задание интервала\n");
        exit(1);
```

```
    }
    if (Eps <= 0.0) {
        puts("Неверное задание точности\n");
        exit(1);
    }

    N = 0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    while ((Right - Left) >= E)
    {
        X = 0.5 * (Right + Left); /* вычисление середины отрезка */
        Y = F(X);
        if (Y == 0.0) return (X);

        if (Y * FLeft < 0.0)
            Right = X;
        else {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    return (X);
}

double Round(double X, double Delta)
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0) return (Delta * (long((X / Delta) + 0.5)));
    else         return (Delta * (long((X / Delta) - 0.5)));
}

double ITER(double X0, double Eps, int &N)
{
    if (Eps <= 0.0) {puts("Неверное задание точности\n"); exit (1);}
    double X1 = F(X0);
    double X2 = F(X1);
    N = 2;
    while( (X1 - X2)*(X1 - X2) > fabs((2*X1 - X0 - X2)*Eps) )
    {
        X0 = X1;
        X1 = X2;
        X2 = F(X1);
        N++;
    }
    return(X2);
}

#ifdef __NEWTON
double NEWTON (double X, double Eps, int &N)
{
    extern double F1 (double);
    double Y, Y1, DX;
    N = 0;
    do
    {
        Y = F(X);
        if (Y == 0.0) return (X);
```

```
        Y1 = F1(X);
        if (Y1 == 0.0) {puts("Производная обратилась в ноль\n"); exit(1);}
        DX = Y / Y1; X = X - DX; N++;
    }
    while (fabs(DX) > Eps);
    return (X);
}
#endif

double HORDA(double Left, double Right, double Eps, int &N)
{
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;
    if (FLeft * FRight > 0.0) {puts("Неверное задание интервала\n"); exit(1);}
    if (Eps <= 0.0) {puts("Неверное задание точности\n"); exit(1);}
    N = 0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;
    do
    {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);
        if (Y == 0.0) return (X);
        if (Y * FLeft < 0.0)
        { Right = X; FRight = Y; }
        else
        { Left = X; FLeft = Y; }
        N++;
    }
    while ( fabs(Y) >= Eps );
    return(X);
}
```

Файл methods.hpp

```
#ifndef METHODS_HPP
#define METHODS_HPP

extern double F(double);

#ifdef __NEWTON
extern double F1(double);
#endif

double Round(double X, double Delta);
double BISECT(double Left, double Right, double Eps, int &N);
double ITER(double X0, double Eps, int &N);
double HORDA(double Left, double Right, double Eps, int &N);
double NEWTON(double X, double Eps, int &N);

#endif // METHODS_HPP
```